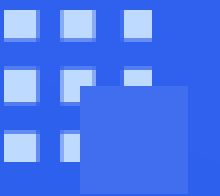




软件测试

日期程序NextDay



日期程序 NextDay 需求规格说明

输入需要计算的日期判断用户输入的日期是否合法，包括年份是否规定范围，例如在 1-9999 之间、月份是否在 1-12 之间、日期是否在 1-31 之间。

计算下一天日期：根据输入的日期计算出下一天的日期。

NextDay 程序是三个 变量的函数：year、month、day，它返回输入日期之后的下一个日期：year、month、day 三个变量，规格条件如下所示。

① $1 \leq \text{year} \leq 9999$

② $1 \leq \text{month} \leq 12$

③ $1 \leq \text{day} \leq 31$

④ 输出日期为下一天

思考与讨论

需求规格的精确性

需求规格的完备性

隐含的需求规格

NextDay 程序中的复杂性主要来自于闰年判断。闰年的发现最早可以追溯到南北朝的数学家祖冲之。如果每四年计算一次闰年，则会出现微小错误。因此通过调整世纪年的闰年来解决。根据上述隐含需求规格，来实现具体的闰年判断。

闰年判断的子程序

一年的时间:

大约为365.2422天

闰年规则:

每四年一次，但世纪年需是 400 的倍数。示例：1980 年和1996 年是闰年，因为它们是4的倍数且不是世纪年。 2000 年是闰年，因为它是世纪年且是400的倍数，但 1900 年不是闰年，因为它是世纪年但不是400的倍数。

```
def LeapYear(year):  
    if year % 400 == 0:  
        return 1  
    if year % 100 == 0:  
        return 0  
    if year % 4 == 0:  
        return 1  
    else:  
        return 0
```

```
def LeapYear(year):  
    if year % 4 == 0:  
        if year % 400 == 0:  
            return 1  
        elif year % 100 == 0:  
            return 0  
        else:  
            return 1  
    else:  
        return 0
```

思考与讨论:

1. 上述两者写法哪个正确？尝试编写其他方式。
2. LeapYear函数的分支特征和Triangle的区别？

```
def next_day(year, month, day):
    tomorrow_month, tomorrow_day, tomorrow_year = month, day, year
    if month in [1, 3, 5, 7, 8, 10]:
        if day < 31:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = month + 1
    elif month in [4, 6, 9, 11]:
        if day < 30:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = month + 1
    elif month == 12:
        if day < 31:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = 1
            tomorrow_year = year + 1
```

```
    elif month == 2:
        if day < 28:
            tomorrow_day = day + 1
        elif day == 28:
            if leap_year(year) == 1:
                tomorrow_day = 29
            else:
                tomorrow_day = 1
                tomorrow_month = 3
        elif day == 29:
            tomorrow_day = 1
            tomorrow_month = 3
    return (tomorrow_year, tomorrow_month, tomorrow_day)
```

思考与讨论:

1. 分支+计算程序的注意事项。
2. 尝试编写其他方法的日期计算。

测试设计示例

用例编号	year	month	day	预期输出	实际输出
1	2007	9	25	2007/9/26	
2	2010	5	3	2010/5/4	
3	2022	12	31	2023/1/1	
4	2001	2	28	2001/3/1	
5	2006	6	30	2006/7/1	
6	2001	2	29	无效日期	
7	2006	6	31	无效日期	

思考与讨论：边界是否考虑充分？

回顾与讨论

讨论：能否给定有限的测试确定一个NextDay程序？



测试设计与测试策略

- 分支与计算结合的测试
- 等价类与边界值的结合
- 控制流与数据流的结合

课后练习：如何测试农历与公历的转换程序？